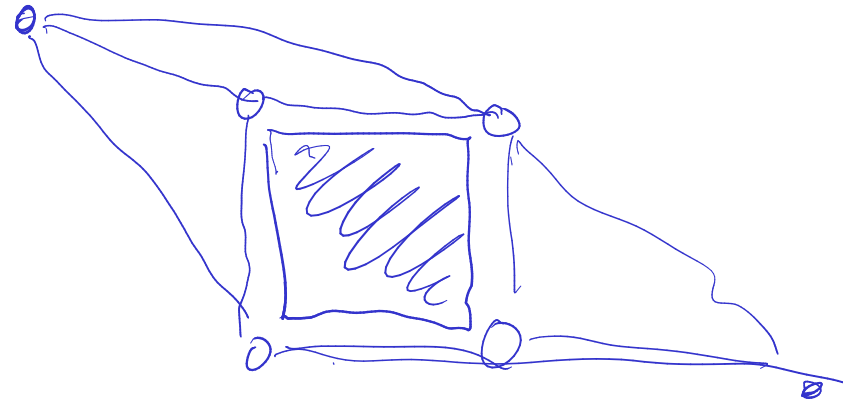


Mein Vorschlag

1. Benutze Hightower's Algorithm: Schnell, eher gerade Verbindung, nicht die kürzesten und findet nicht immer eine.
Sollte also gut passen für unsere Anwendung. Behalte A* als Fallback.
2. Vermeide Neuberechnungen: Berechne nur Pfade neu, wenn etwas innerhalb ihrer Bounding Box geändert wurde.
3. Falls der A* Fall noch Regelmäßig getriggert wird: Micro-Optimierung der Implementierung
4. Wenn das nicht ausreicht, mglw. ausgefeiltere Erweiterungen von A* (bspw. Visibility Graph oder Landmark Precomputation)

Stichwörter für weitere Recherche

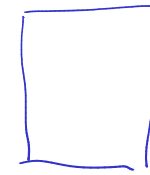
- * PCB Routing
- * Gamedev: Pathfinding
- * Graph Layout



Hightowers Algorithm

- * nicht shortest
- * sehr simpel
- * benötigt nur die Linien der Hindernisse (kein Gitter)
- * findet nicht unbedingt einen Pfad, auch wenn einer existiert

Artikel: <https://dl.acm.org/doi/pdf/10.1145/800260.809014>

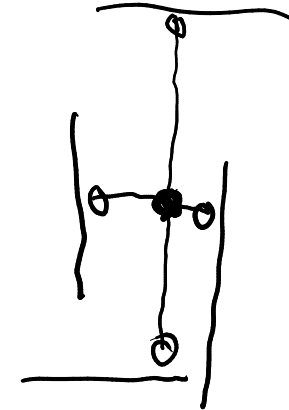
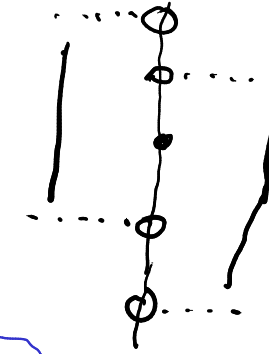
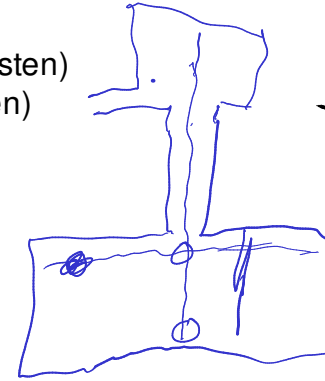


Aus PCB Routing

Auch bekannt als "Line Probing", "Line Searching"

Verwandte Algos: Mikami and Tabuchi, Track Graph

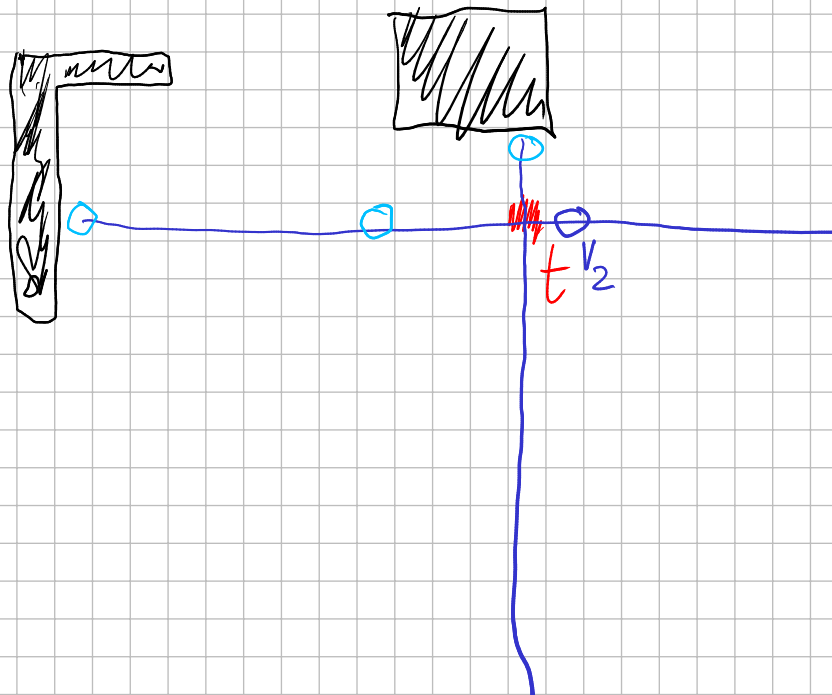
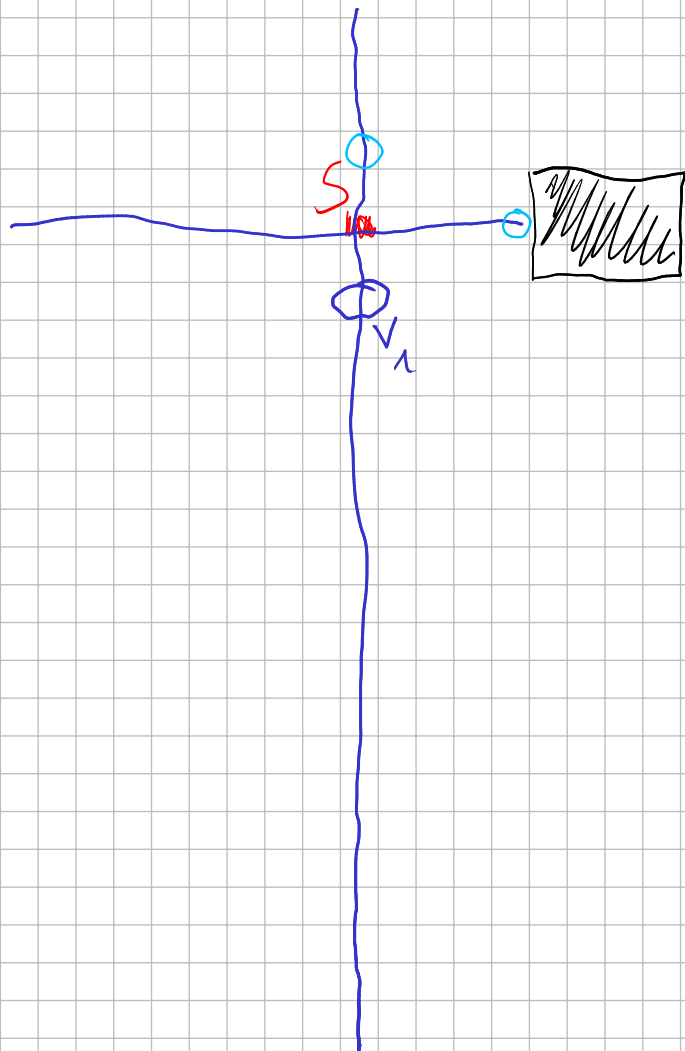
- * horizontale und vertikale "escape lines" durch "escape points"
- * abwechselnd vom Startpunkt s und Zielpunkt t aus
- * finde einen escape point
 - * entweder der Punkt, der über ein "Cover" übersteht (wähle den euklisch nächsten)
 - * oder der Punkt vor einer Kollision mit dem Cover (wähle den euklisch nächsten)
- * sobald sich escape lines von s und t treffen -> Pfad gefunden
- * optional: Pfadvereinfachung



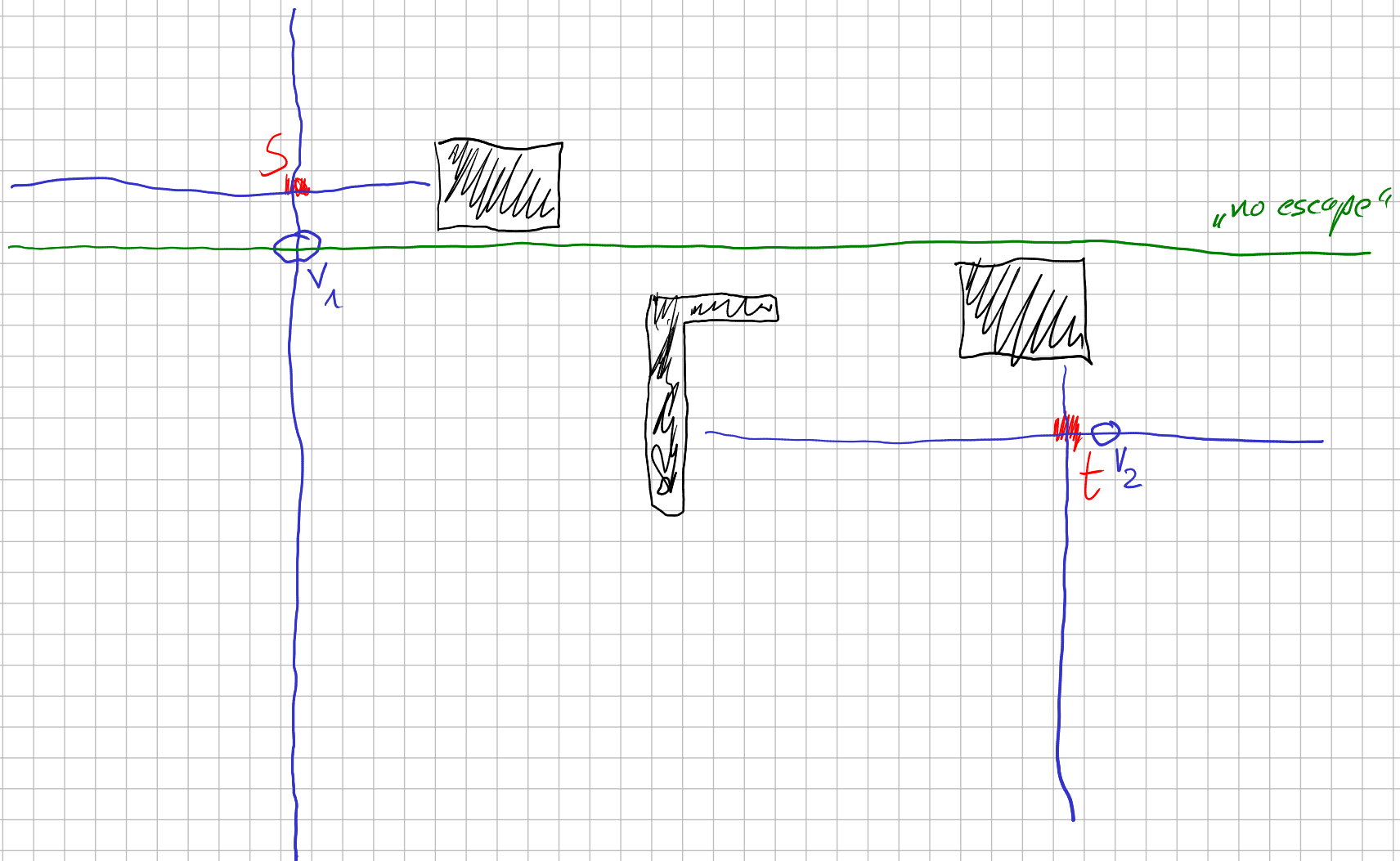
S



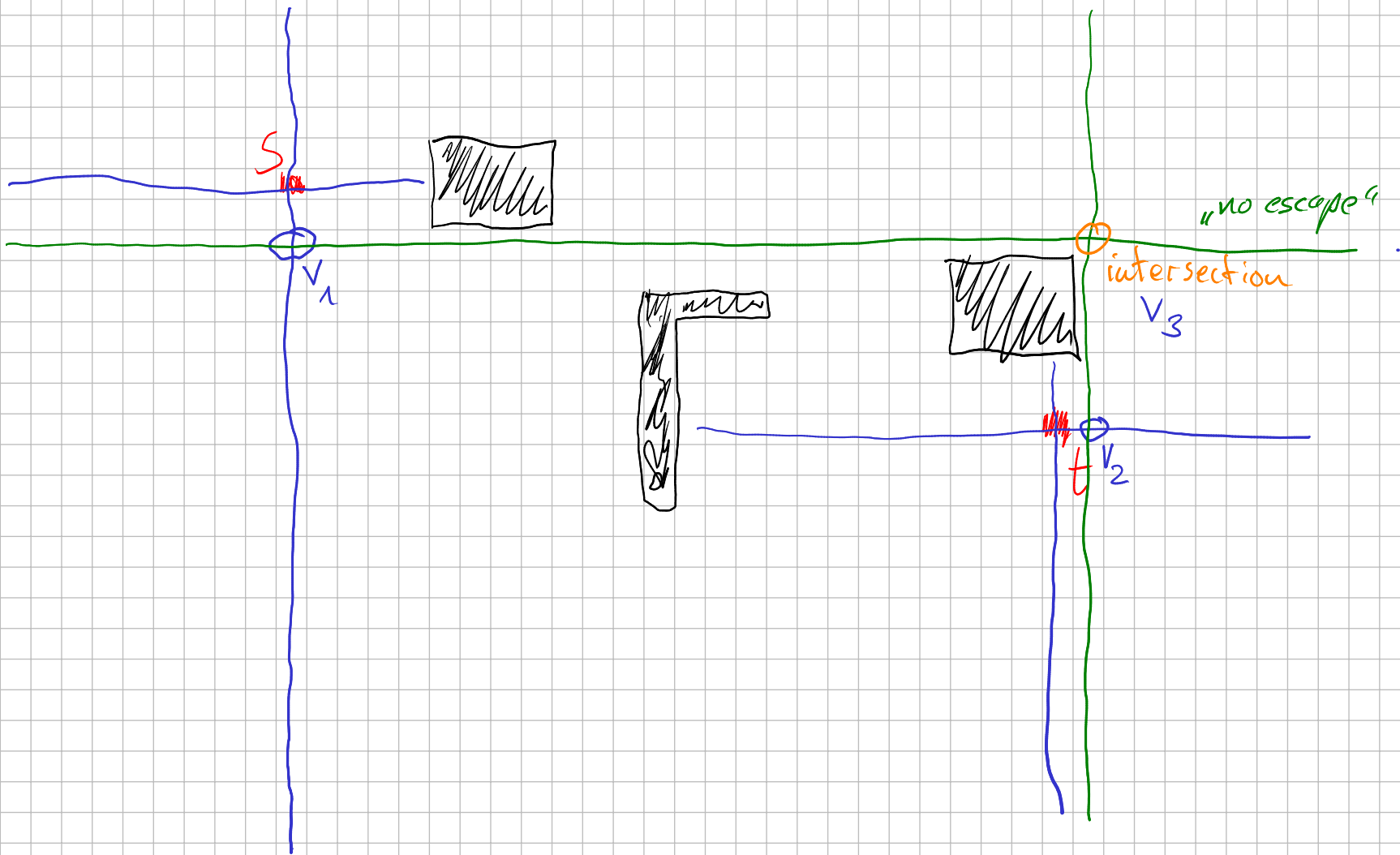
t



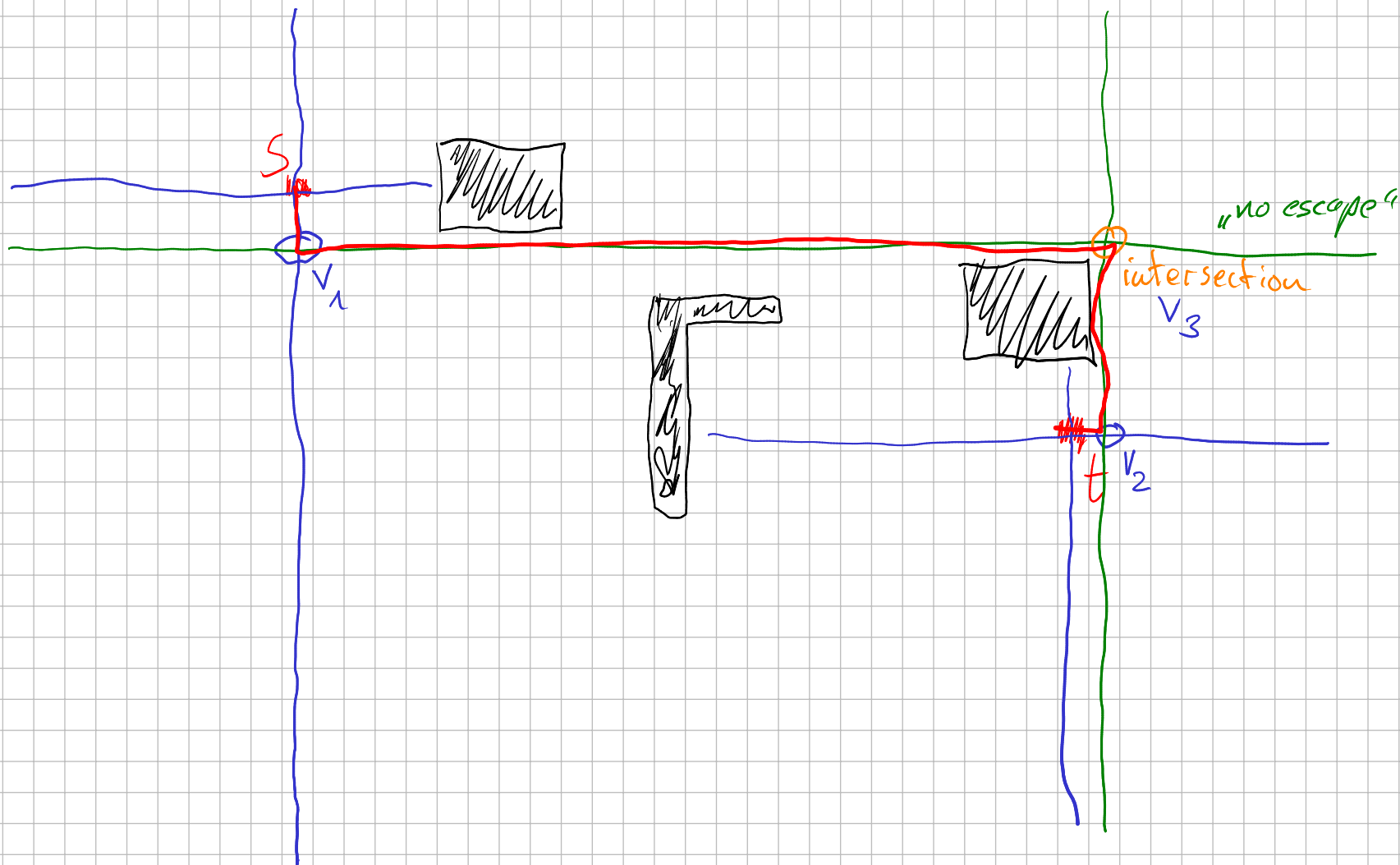
L_s	L_t
S	t
v_1	v_2



L_s	L_t
S	t
V_1	V_2



L_S	L_E
S	t
v_1	v_2
	v_3



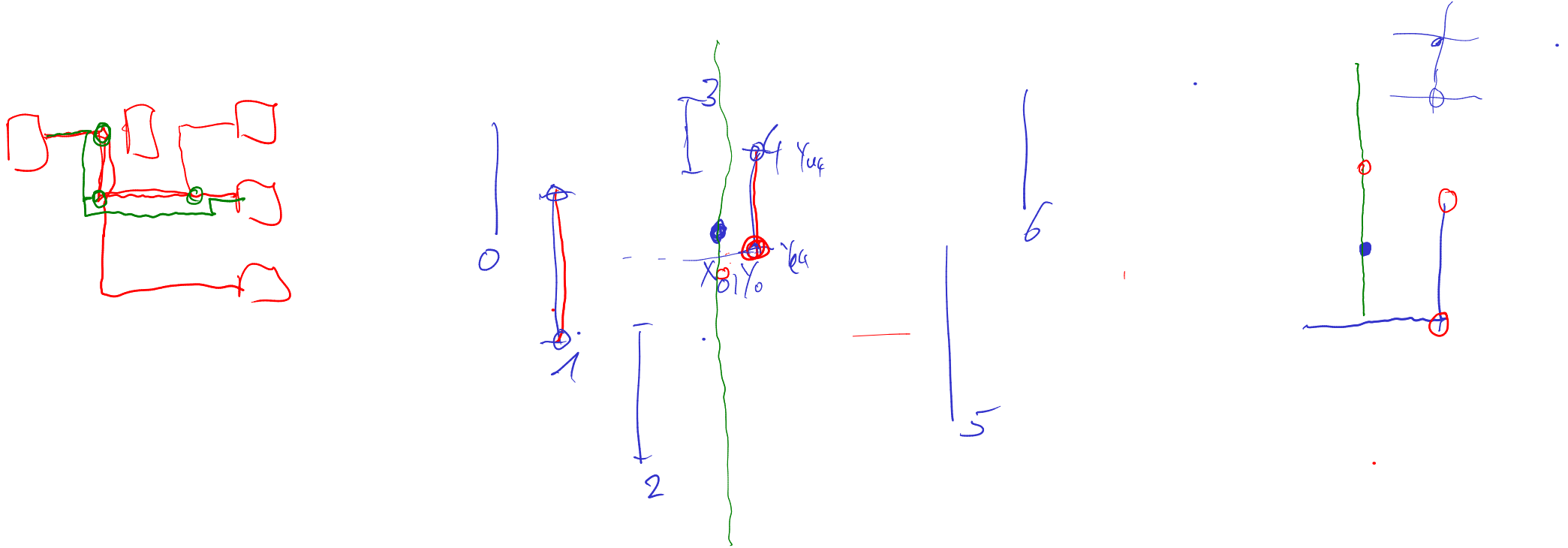
L_S	L_t
S	t
v_1	v_2
	v_3

"Path Determining Algorithm"

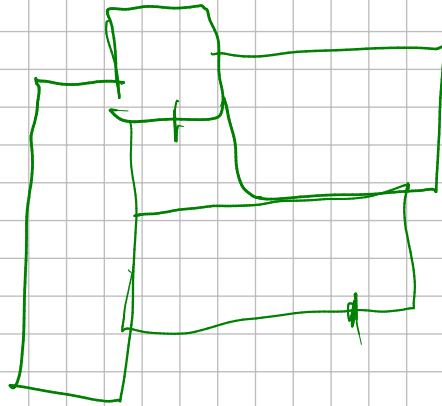
S, v_1, v_3, v_2, t

Anmerkungen

- * Cover finden: Listen für jeweils horizontale bzw. vertikale Kanten
Sortiert nach y- bzw. x-Koordinate
-> finde Position vom aktuellen Escape Point (Object Point) per Bisection
gehe von da aus aufwärts und abwärts bis die x- bzw. y-Koordinate innerhalb der Kante ist, um die Cover zu finden
- * Vermeide Pfad-Überlappungen: füge Eckpunkte des gefundenen Pfades als "kurze Kanten" zum Problem hinzu
-> Kreuzungen erlaubt, aber lange übereinander laufende Linien nicht



.



Erweiterung denkbar

→ mehr als einen Escape-Point
dann Backtracking

